In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import roc_curve, auc, confusion_matrix
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
from sklearn.preprocessing import label_binarize
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import transforms
import torch.utils.data as data
import os
from torch.cuda.amp import autocast, GradScaler
from torch.nn.utils import clip_grad_norm_
import timm
import math
from torch.optim.lr_scheduler import LambdaLR



import medmnist
from medmnist import INFO, Evaluator
```

In [2]:
```
data_flag = 'dermamnist'  # Set as dermamnist
download = True

NUM_EPOCHS = 3
BATCH_SIZE = 128
lr = 0.001

# INFO is a dictionary that contains information
# about all datasets
info = INFO[data_flag]
#Retrieve information from the dermamnist dataset
task = info['task']
n_channels = info['n_channels']
n_classes = len(info['label'])
num_classes = len(info['label'])

#Get data class
DataClass = getattr(medmnist, info['python_class'])
```

Dataloading

In [3]:
```
# preprocessing
data_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[.5], std=[.5])
])

# load the data
train_dataset = DataClass(split='train',
                          transform=data_transform,
                          download=download,
                          size=64, mmap_mode='r')
test_dataset = DataClass(split='test',
                         transform=data_transform,
```

```
                          download=download,
                          size=64, mmap_mode='r')
val_dataset = DataClass(split='val',
                        transform=data_transform,
                        download=download,
                        size=64, mmap_mode='r')

# 设置基本参数
IMG_SIZE = 64

# encapsulate data into dataloader form
train_loader = data.DataLoader(dataset=train_dataset,
                        batch_size=BATCH_SIZE, shuffle=True)
val_loader = data.DataLoader(dataset=val_dataset,
                        batch_size=BATCH_SIZE, shuffle=True)
test_loader = data.DataLoader(dataset=test_dataset,
                        batch_size=BATCH_SIZE, shuffle=True)

# 打印数据集信息
print(f"Train size: {len(train_loader.dataset)}")
print(f"Val size: {len(val_loader.dataset)}")
print(f"Test size: {len(test_loader.dataset)}")
print(f"BATCH_SIZE: {BATCH_SIZE}")
print(f"IMG_SIZE: {IMG_SIZE}x{IMG_SIZE}")
```

```
Using downloaded and verified file: C:\Users\Park\.medmnist\dermamnist_64.npz
Using downloaded and verified file: C:\Users\Park\.medmnist\dermamnist_64.npz
Using downloaded and verified file: C:\Users\Park\.medmnist\dermamnist_64.npz
Train size: 7007
Val size: 1003
Test size: 2005
BATCH_SIZE: 128
IMG_SIZE: 64x64
```

Preprocessing: Chang the datasize from 64 to 224

In [4]:

```
IMG_SIZE = 224
# Data transformations
data_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE), interpolation=transforms.Interpolati
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    data_transform
])

train_dataset.transform = train_transform
val_dataset.transform = data_transform
test_dataset.transform = data_transform

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=BATCH_SIZE,
val_loader = torch.utils.data.DataLoader(val_dataset, batch_size=BATCH_SIZE, shu
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=BATCH_SIZE, s
```

Data processing

In [5]:
```
# Data augmentation and transformation
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# Apply transformations to datasets
train_dataset.transform = train_transform
val_dataset.transform = val_transform
test_dataset.transform = test_transform

# Create data loaders
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=4,
    pin_memory=True
)
val_loader = torch.utils.data.DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=4,
    pin_memory=True
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=4,
    pin_memory=True
)
```

Vision Transformer -- Vit_tiny_patch16_224

Define model

In [7]:
```
# Set training parameters
NUM_EPOCHS = 200
BATCH_SIZE = 64
lr = 3e-4
GRAD_CLIP = 5.0
PATIENCE = 25
WARMUP_EPOCHS = 5
IMG_SIZE = 224
save_path = "Skin Disease Transformer"

# Ensure save directory exists
os.makedirs(save_path, exist_ok=True)

# Custom Label Smoothing Loss
class LabelSmoothingCrossEntropy(nn.Module):
    def __init__(self, smoothing=0.1):
        super().__init__()
        self.smoothing = smoothing

    def forward(self, x, target):
        confidence = 1. - self.smoothing
        logprobs = torch.nn.functional.log_softmax(x, dim=-1)
        nll_loss = -logprobs.gather(dim=-1, index=target.unsqueeze(1))
        nll_loss = nll_loss.squeeze(1)
        smooth_loss = -logprobs.mean(dim=-1)
        loss = confidence * nll_loss + self.smoothing * smooth_loss
        return loss.mean()


# Vit_tiny_patch16_224
model_name = "vit_tiny_patch16_224"
model = timm.create_model(model_name, num_classes=n_classes, pretrained=True).cu

# Define optimizer and loss
optimizer = optim.AdamW(
    model.parameters(),
    lr=lr,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0.05
)

criterion = LabelSmoothingCrossEntropy(smoothing=0.1).cuda()

# Learning rate scheduler with warmup
total_steps = NUM_EPOCHS * len(train_loader)
warmup_steps = WARMUP_EPOCHS * len(train_loader)

def get_cosine_schedule_with_warmup(optimizer, num_warmup_steps, num_training_st
    def lr_lambda(current_step):
        if current_step < num_warmup_steps:
            return float(current_step) / float(max(1, num_warmup_steps))
        progress = float(current_step - num_warmup_steps) / float(max(1, num_tra
        return max(min_lr, 0.5 * (1.0 + math.cos(math.pi * progress)))

    return LambdaLR(optimizer, lr_lambda)

scheduler = get_cosine_schedule_with_warmup(optimizer, warmup_steps, total_steps
```

```
# Mixup function
def mixup_data(x, y, alpha=0.2):
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size()[0]
    index = torch.randperm(batch_size).cuda()

    mixed_x = lam * x + (1 - lam) * x[index, :]
    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam

# Initialize training components
scaler = GradScaler()
best_val_acc = 0.0
best_val_loss = float('inf')
early_stopping_counter = 0
```

```
C:\Users\Park\AppData\Local\Temp\ipykernel_75228\748525688.py:88: FutureWarning:
`torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradSca
ler('cuda', args...)` instead.
  scaler = GradScaler()
```

Training Phase

In [16]:

```
# Training loop
for epoch in range(NUM_EPOCHS):
    # Training phase
    model.train()
    train_correct = 0
    train_total = 0
    train_loss = 0.0

    for inputs, targets in train_loader:
        inputs = inputs.cuda()
        targets = targets.cuda().squeeze().long()

        # Apply Mixup
        inputs, targets_a, targets_b, lam = mixup_data(inputs, targets)

        optimizer.zero_grad(set_to_none=True)

        with autocast():
            outputs = model(inputs)
            loss = lam * criterion(outputs, targets_a) + (1 - lam) * criterion(c
            _, predictions = torch.max(outputs, 1)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        clip_grad_norm_(model.parameters(), GRAD_CLIP)
        scaler.step(optimizer)
        scaler.update()
        scheduler.step()

        train_total += targets.size(0)
        train_correct += (predictions == targets).sum().item()
        train_loss += loss.item()
```

```
    train_epoch_loss = train_loss / len(train_loader)
    train_epoch_acc = 100 * train_correct / train_total

    # Validation phase
    model.eval()
    val_correct = 0
    val_total = 0
    val_loss = 0.0

    with torch.no_grad():
        for inputs, targets in val_loader:
            inputs = inputs.cuda()
            targets = targets.cuda().squeeze().long()

            with autocast():
                outputs = model(inputs)
                loss = criterion(outputs, targets)
                _, predictions = torch.max(outputs, 1)

            val_total += targets.size(0)
            val_correct += (predictions == targets).sum().item()
            val_loss += loss.item()

    val_epoch_loss = val_loss / len(val_loader)
    val_acc = 100 * val_correct / val_total

    # Print epoch results
    print(
        f'Epoch [{epoch+1}/{NUM_EPOCHS}] '
        f'Train Loss: {train_epoch_loss:.4f} Acc: {train_epoch_acc:.2f}% | '
        f'Val Loss: {val_epoch_loss:.4f} Acc: {val_acc:.2f}% | '
        f'LR: {scheduler.get_last_lr()[0]:.6f}'
    )

    # Save best model based on validation accuracy
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_val_loss = val_epoch_loss

        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'scheduler_state_dict': scheduler.state_dict(),
            'val_acc': val_acc,
            'val_loss': val_epoch_loss,
            'scaler': scaler.state_dict(),
        }, os.path.join(save_path, 'Vit_best_model.pth'))

        print(f'Model saved (Val Acc: {val_acc:.2f}%, Val Loss: {val_epoch_loss:
        early_stopping_counter = 0
    else:
        early_stopping_counter += 1

    # Early stopping check
    if early_stopping_counter >= PATIENCE:
        print(f'Early stopping triggered after {epoch+1} epochs')
        break

# Load best model
```

```
print(f'Loading best model (Val Acc: {best_val_acc:.2f}%)')
checkpoint = torch.load(os.path.join(save_path, 'Vit_best_model.pth'))
model.load_state_dict(checkpoint['model_state_dict'])
```

```
C:\Users\Park\PycharmProjects\pythonProject\.venv\Lib\site-packages\huggingface_h
ub\file_download.py:140: UserWarning: `huggingface_hub` cache-system uses symlink
s by default to efficiently store duplicated files but your machine does not supp
ort them in C:\Users\Park\.cache\huggingface\hub\models--timm--vit_tiny_patch16_2
24.augreg_in21k_ft_in1k. Caching files will still work but in a degraded version
that might require more space on your disk. This warning can be disabled by setti
ng the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details,
see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to
run Python as an administrator. In order to activate developer mode, see this art
icle: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-devic
e-for-development
  warnings.warn(message)
C:\Users\Park\AppData\Local\Temp\ipykernel_58780\2218515974.py:125: FutureWarnin
g: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.Grad
Scaler('cuda', args...)` instead.
  scaler = GradScaler()
C:\Users\Park\AppData\Local\Temp\ipykernel_58780\2218515974.py:147: FutureWarnin
g: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autoca
st('cuda', args...)` instead.
  with autocast():
C:\Users\Park\PycharmProjects\pythonProject\.venv\Lib\site-packages\torch\optim\l
r_scheduler.py:224: UserWarning: Detected call of `lr_scheduler.step()` before `o
ptimizer.step()`. In PyTorch 1.1.0 and later, you should call them in the opposit
e order: `optimizer.step()` before `lr_scheduler.step()`.  Failure to do this wil
l result in PyTorch skipping the first value of the learning rate schedule. See m
ore details at https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-
rate
  warnings.warn(
C:\Users\Park\AppData\Local\Temp\ipykernel_58780\2218515974.py:177: FutureWarnin
g: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autoca
st('cuda', args...)` instead.
  with autocast():
```

```
Epoch [1/200] Train Loss: 1.3359 Acc: 56.61% | Val Loss: 0.9871 Acc: 73.38% | LR:
0.000060
Model saved (Val Acc: 73.38%, Val Loss: 0.9871)
Epoch [2/200] Train Loss: 1.0197 Acc: 66.28% | Val Loss: 0.9244 Acc: 78.66% | LR:
0.000120
Model saved (Val Acc: 78.66%, Val Loss: 0.9244)
Epoch [3/200] Train Loss: 0.9844 Acc: 65.29% | Val Loss: 0.8467 Acc: 81.66% | LR:
0.000180
Model saved (Val Acc: 81.66%, Val Loss: 0.8467)
Epoch [4/200] Train Loss: 0.9600 Acc: 67.59% | Val Loss: 0.8616 Acc: 79.96% | LR:
0.000240
Epoch [5/200] Train Loss: 0.9520 Acc: 68.00% | Val Loss: 0.8426 Acc: 82.45% | LR:
0.000300
Model saved (Val Acc: 82.45%, Val Loss: 0.8426)
Epoch [6/200] Train Loss: 0.9318 Acc: 66.60% | Val Loss: 0.8361 Acc: 82.65% | LR:
0.000300
Model saved (Val Acc: 82.65%, Val Loss: 0.8361)
Epoch [7/200] Train Loss: 0.9120 Acc: 68.40% | Val Loss: 0.8097 Acc: 82.75% | LR:
0.000300
Model saved (Val Acc: 82.75%, Val Loss: 0.8097)
Epoch [8/200] Train Loss: 0.8857 Acc: 68.97% | Val Loss: 0.8352 Acc: 80.06% | LR:
0.000300
Epoch [9/200] Train Loss: 0.8887 Acc: 67.55% | Val Loss: 0.8027 Acc: 83.35% | LR:
0.000300
Model saved (Val Acc: 83.35%, Val Loss: 0.8027)
Epoch [10/200] Train Loss: 0.8750 Acc: 69.13% | Val Loss: 0.8194 Acc: 81.66% | L
R: 0.000300
Epoch [11/200] Train Loss: 0.8485 Acc: 66.78% | Val Loss: 0.7752 Acc: 84.95% | L
R: 0.000299
Model saved (Val Acc: 84.95%, Val Loss: 0.7752)
Epoch [12/200] Train Loss: 0.8457 Acc: 68.62% | Val Loss: 0.7847 Acc: 84.15% | L
R: 0.000299
Epoch [13/200] Train Loss: 0.8173 Acc: 66.95% | Val Loss: 0.8370 Acc: 82.55% | L
R: 0.000299
Epoch [14/200] Train Loss: 0.8320 Acc: 65.69% | Val Loss: 0.8089 Acc: 82.95% | L
R: 0.000298
Epoch [15/200] Train Loss: 0.8252 Acc: 68.10% | Val Loss: 0.7795 Acc: 84.75% | L
R: 0.000298
Epoch [16/200] Train Loss: 0.8327 Acc: 67.22% | Val Loss: 0.7722 Acc: 85.04% | L
R: 0.000298
Model saved (Val Acc: 85.04%, Val Loss: 0.7722)
Epoch [17/200] Train Loss: 0.7864 Acc: 69.59% | Val Loss: 0.8012 Acc: 83.05% | L
R: 0.000297
Epoch [18/200] Train Loss: 0.8077 Acc: 70.50% | Val Loss: 0.8385 Acc: 83.35% | L
R: 0.000297
Epoch [19/200] Train Loss: 0.7799 Acc: 69.53% | Val Loss: 0.7825 Acc: 84.05% | L
R: 0.000296
Epoch [20/200] Train Loss: 0.7743 Acc: 72.16% | Val Loss: 0.7995 Acc: 83.75% | L
R: 0.000296
Epoch [21/200] Train Loss: 0.7390 Acc: 71.74% | Val Loss: 0.8152 Acc: 84.25% | L
R: 0.000295
Epoch [22/200] Train Loss: 0.7626 Acc: 70.20% | Val Loss: 0.8151 Acc: 82.75% | L
R: 0.000294
Epoch [23/200] Train Loss: 0.7804 Acc: 74.55% | Val Loss: 0.7599 Acc: 85.24% | L
R: 0.000294
Model saved (Val Acc: 85.24%, Val Loss: 0.7599)
Epoch [24/200] Train Loss: 0.7596 Acc: 70.34% | Val Loss: 0.7531 Acc: 86.24% | L
R: 0.000293
Model saved (Val Acc: 86.24%, Val Loss: 0.7531)
Epoch [25/200] Train Loss: 0.7232 Acc: 67.25% | Val Loss: 0.7996 Acc: 83.75% | L
```

```
R: 0.000292
Epoch [26/200] Train Loss: 0.7273 Acc: 71.27% | Val Loss: 0.7701 Acc: 85.44% | L
R: 0.000291
Epoch [27/200] Train Loss: 0.7469 Acc: 72.13% | Val Loss: 0.7863 Acc: 84.45% | L
R: 0.000291
Epoch [28/200] Train Loss: 0.7540 Acc: 70.69% | Val Loss: 0.7927 Acc: 85.24% | L
R: 0.000290
Epoch [29/200] Train Loss: 0.7322 Acc: 68.77% | Val Loss: 0.7870 Acc: 84.45% | L
R: 0.000289
Epoch [30/200] Train Loss: 0.7637 Acc: 70.23% | Val Loss: 0.7888 Acc: 84.75% | L
R: 0.000288
Epoch [31/200] Train Loss: 0.6793 Acc: 70.84% | Val Loss: 0.7690 Acc: 84.15% | L
R: 0.000287
Epoch [32/200] Train Loss: 0.7458 Acc: 71.74% | Val Loss: 0.7982 Acc: 83.65% | L
R: 0.000286
Epoch [33/200] Train Loss: 0.7268 Acc: 71.77% | Val Loss: 0.7684 Acc: 85.04% | L
R: 0.000285
Epoch [34/200] Train Loss: 0.7090 Acc: 70.26% | Val Loss: 0.7849 Acc: 84.05% | L
R: 0.000284
Epoch [35/200] Train Loss: 0.6915 Acc: 73.68% | Val Loss: 0.7855 Acc: 85.04% | L
R: 0.000283
Epoch [36/200] Train Loss: 0.6894 Acc: 73.88% | Val Loss: 0.7739 Acc: 85.54% | L
R: 0.000282
Epoch [37/200] Train Loss: 0.6870 Acc: 77.09% | Val Loss: 0.8026 Acc: 83.95% | L
R: 0.000281
Epoch [38/200] Train Loss: 0.6936 Acc: 72.31% | Val Loss: 0.7749 Acc: 85.54% | L
R: 0.000279
Epoch [39/200] Train Loss: 0.7089 Acc: 70.56% | Val Loss: 0.7856 Acc: 85.14% | L
R: 0.000278
Epoch [40/200] Train Loss: 0.6847 Acc: 73.07% | Val Loss: 0.7601 Acc: 85.94% | L
R: 0.000277
Epoch [41/200] Train Loss: 0.6811 Acc: 69.74% | Val Loss: 0.8095 Acc: 83.05% | L
R: 0.000275
Epoch [42/200] Train Loss: 0.6820 Acc: 69.86% | Val Loss: 0.8046 Acc: 84.25% | L
R: 0.000274
Epoch [43/200] Train Loss: 0.6690 Acc: 73.00% | Val Loss: 0.8138 Acc: 83.65% | L
R: 0.000273
Epoch [44/200] Train Loss: 0.6491 Acc: 72.93% | Val Loss: 0.7806 Acc: 85.44% | L
R: 0.000271
Epoch [45/200] Train Loss: 0.6733 Acc: 68.72% | Val Loss: 0.8086 Acc: 83.75% | L
R: 0.000270
Epoch [46/200] Train Loss: 0.6680 Acc: 66.65% | Val Loss: 0.7973 Acc: 84.95% | L
R: 0.000268
Epoch [47/200] Train Loss: 0.6623 Acc: 72.41% | Val Loss: 0.8033 Acc: 85.04% | L
R: 0.000267
Epoch [48/200] Train Loss: 0.6796 Acc: 73.47% | Val Loss: 0.7699 Acc: 86.54% | L
R: 0.000265
Model saved (Val Acc: 86.54%, Val Loss: 0.7699)
Epoch [49/200] Train Loss: 0.6875 Acc: 70.49% | Val Loss: 0.7648 Acc: 85.44% | L
R: 0.000264
Epoch [50/200] Train Loss: 0.6581 Acc: 70.33% | Val Loss: 0.7708 Acc: 85.54% | L
R: 0.000262
Epoch [51/200] Train Loss: 0.6749 Acc: 74.24% | Val Loss: 0.7693 Acc: 85.34% | L
R: 0.000261
Epoch [52/200] Train Loss: 0.6977 Acc: 70.00% | Val Loss: 0.7920 Acc: 85.34% | L
R: 0.000259
Epoch [53/200] Train Loss: 0.6766 Acc: 71.66% | Val Loss: 0.7881 Acc: 85.54% | L
R: 0.000257
Epoch [54/200] Train Loss: 0.6828 Acc: 75.40% | Val Loss: 0.7622 Acc: 85.64% | L
R: 0.000256
```

```
Epoch [55/200] Train Loss: 0.6677 Acc: 75.72% | Val Loss: 0.7893 Acc: 85.04% | L
R: 0.000254
Epoch [56/200] Train Loss: 0.6578 Acc: 72.40% | Val Loss: 0.7776 Acc: 85.64% | L
R: 0.000252
Epoch [57/200] Train Loss: 0.6675 Acc: 76.68% | Val Loss: 0.7891 Acc: 84.75% | L
R: 0.000250
Epoch [58/200] Train Loss: 0.6082 Acc: 74.37% | Val Loss: 0.7929 Acc: 84.85% | L
R: 0.000249
Epoch [59/200] Train Loss: 0.6852 Acc: 71.19% | Val Loss: 0.8000 Acc: 85.54% | L
R: 0.000247
Epoch [60/200] Train Loss: 0.6569 Acc: 72.31% | Val Loss: 0.7925 Acc: 85.34% | L
R: 0.000245
Epoch [61/200] Train Loss: 0.6337 Acc: 72.26% | Val Loss: 0.7903 Acc: 86.14% | L
R: 0.000243
Epoch [62/200] Train Loss: 0.6270 Acc: 70.69% | Val Loss: 0.8277 Acc: 84.45% | L
R: 0.000241
Epoch [63/200] Train Loss: 0.6902 Acc: 72.09% | Val Loss: 0.7867 Acc: 85.04% | L
R: 0.000239
Epoch [64/200] Train Loss: 0.6638 Acc: 74.91% | Val Loss: 0.8087 Acc: 85.04% | L
R: 0.000237
Epoch [65/200] Train Loss: 0.6202 Acc: 72.27% | Val Loss: 0.8028 Acc: 85.84% | L
R: 0.000235
Epoch [66/200] Train Loss: 0.6325 Acc: 71.24% | Val Loss: 0.7898 Acc: 85.04% | L
R: 0.000233
Epoch [67/200] Train Loss: 0.6285 Acc: 75.97% | Val Loss: 0.8165 Acc: 85.44% | L
R: 0.000231
Epoch [68/200] Train Loss: 0.6297 Acc: 70.76% | Val Loss: 0.7804 Acc: 86.84% | L
R: 0.000229
Model saved (Val Acc: 86.84%, Val Loss: 0.7804)
Epoch [69/200] Train Loss: 0.6140 Acc: 74.53% | Val Loss: 0.8095 Acc: 84.75% | L
R: 0.000227
Epoch [70/200] Train Loss: 0.6176 Acc: 70.62% | Val Loss: 0.7818 Acc: 85.84% | L
R: 0.000225
Epoch [71/200] Train Loss: 0.6286 Acc: 73.50% | Val Loss: 0.8007 Acc: 85.34% | L
R: 0.000223
Epoch [72/200] Train Loss: 0.6325 Acc: 70.42% | Val Loss: 0.7742 Acc: 86.44% | L
R: 0.000221
Epoch [73/200] Train Loss: 0.6438 Acc: 71.53% | Val Loss: 0.7601 Acc: 87.34% | L
R: 0.000219
Model saved (Val Acc: 87.34%, Val Loss: 0.7601)
Epoch [74/200] Train Loss: 0.6115 Acc: 70.79% | Val Loss: 0.7749 Acc: 87.04% | L
R: 0.000216
Epoch [75/200] Train Loss: 0.6100 Acc: 75.71% | Val Loss: 0.8166 Acc: 86.14% | L
R: 0.000214
Epoch [76/200] Train Loss: 0.6216 Acc: 75.30% | Val Loss: 0.7663 Acc: 86.54% | L
R: 0.000212
Epoch [77/200] Train Loss: 0.6356 Acc: 76.32% | Val Loss: 0.7973 Acc: 86.64% | L
R: 0.000210
Epoch [78/200] Train Loss: 0.6544 Acc: 76.74% | Val Loss: 0.7975 Acc: 85.84% | L
R: 0.000208
Epoch [79/200] Train Loss: 0.6193 Acc: 70.84% | Val Loss: 0.7884 Acc: 86.44% | L
R: 0.000205
Epoch [80/200] Train Loss: 0.6446 Acc: 74.88% | Val Loss: 0.7870 Acc: 86.04% | L
R: 0.000203
Epoch [81/200] Train Loss: 0.6105 Acc: 71.13% | Val Loss: 0.7539 Acc: 87.24% | L
R: 0.000201
Epoch [82/200] Train Loss: 0.6106 Acc: 72.34% | Val Loss: 0.7577 Acc: 86.84% | L
R: 0.000199
Epoch [83/200] Train Loss: 0.6024 Acc: 75.07% | Val Loss: 0.7879 Acc: 86.04% | L
R: 0.000196
```

```
Epoch [84/200] Train Loss: 0.6219 Acc: 73.17% | Val Loss: 0.7816 Acc: 85.94% | L
R: 0.000194
Epoch [85/200] Train Loss: 0.6304 Acc: 73.60% | Val Loss: 0.7699 Acc: 86.54% | L
R: 0.000192
Epoch [86/200] Train Loss: 0.6549 Acc: 71.84% | Val Loss: 0.7582 Acc: 87.64% | L
R: 0.000189
Model saved (Val Acc: 87.64%, Val Loss: 0.7582)
Epoch [87/200] Train Loss: 0.6372 Acc: 69.80% | Val Loss: 0.7706 Acc: 87.54% | L
R: 0.000187
Epoch [88/200] Train Loss: 0.6302 Acc: 71.87% | Val Loss: 0.7671 Acc: 87.04% | L
R: 0.000185
Epoch [89/200] Train Loss: 0.6194 Acc: 74.18% | Val Loss: 0.7561 Acc: 87.14% | L
R: 0.000182
Epoch [90/200] Train Loss: 0.6572 Acc: 71.20% | Val Loss: 0.7634 Acc: 86.54% | L
R: 0.000180
Epoch [91/200] Train Loss: 0.6195 Acc: 70.36% | Val Loss: 0.7611 Acc: 87.44% | L
R: 0.000178
Epoch [92/200] Train Loss: 0.6369 Acc: 70.37% | Val Loss: 0.7729 Acc: 87.24% | L
R: 0.000175
Epoch [93/200] Train Loss: 0.6396 Acc: 73.28% | Val Loss: 0.7716 Acc: 85.94% | L
R: 0.000173
Epoch [94/200] Train Loss: 0.6068 Acc: 72.28% | Val Loss: 0.7526 Acc: 87.94% | L
R: 0.000170
Model saved (Val Acc: 87.94%, Val Loss: 0.7526)
Epoch [95/200] Train Loss: 0.6219 Acc: 70.52% | Val Loss: 0.7764 Acc: 85.54% | L
R: 0.000168
Epoch [96/200] Train Loss: 0.6450 Acc: 73.81% | Val Loss: 0.7988 Acc: 87.04% | L
R: 0.000166
Epoch [97/200] Train Loss: 0.6113 Acc: 72.47% | Val Loss: 0.8177 Acc: 84.95% | L
R: 0.000163
Epoch [98/200] Train Loss: 0.6046 Acc: 71.74% | Val Loss: 0.7800 Acc: 87.04% | L
R: 0.000161
Epoch [99/200] Train Loss: 0.6058 Acc: 75.40% | Val Loss: 0.7911 Acc: 85.94% | L
R: 0.000158
Epoch [100/200] Train Loss: 0.6352 Acc: 73.23% | Val Loss: 0.7574 Acc: 87.44% | L
R: 0.000156
Epoch [101/200] Train Loss: 0.5998 Acc: 71.61% | Val Loss: 0.8026 Acc: 86.94% | L
R: 0.000154
Epoch [102/200] Train Loss: 0.6107 Acc: 75.12% | Val Loss: 0.7980 Acc: 86.84% | L
R: 0.000151
Epoch [103/200] Train Loss: 0.6198 Acc: 73.84% | Val Loss: 0.7893 Acc: 86.84% | L
R: 0.000149
Epoch [104/200] Train Loss: 0.5946 Acc: 74.78% | Val Loss: 0.7703 Acc: 87.34% | L
R: 0.000146
Epoch [105/200] Train Loss: 0.6345 Acc: 69.66% | Val Loss: 0.7649 Acc: 87.04% | L
R: 0.000144
Epoch [106/200] Train Loss: 0.5880 Acc: 70.99% | Val Loss: 0.7943 Acc: 86.74% | L
R: 0.000142
Epoch [107/200] Train Loss: 0.6255 Acc: 71.77% | Val Loss: 0.7805 Acc: 87.04% | L
R: 0.000139
Epoch [108/200] Train Loss: 0.6187 Acc: 73.53% | Val Loss: 0.7751 Acc: 87.14% | L
R: 0.000137
Epoch [109/200] Train Loss: 0.6448 Acc: 73.81% | Val Loss: 0.7743 Acc: 86.74% | L
R: 0.000134
Epoch [110/200] Train Loss: 0.5913 Acc: 71.60% | Val Loss: 0.7690 Acc: 85.74% | L
R: 0.000132
Epoch [111/200] Train Loss: 0.5930 Acc: 73.36% | Val Loss: 0.7781 Acc: 87.84% | L
R: 0.000130
Epoch [112/200] Train Loss: 0.5967 Acc: 75.44% | Val Loss: 0.7872 Acc: 86.54% | L
R: 0.000127
```

```
Epoch [113/200] Train Loss: 0.5992 Acc: 72.84% | Val Loss: 0.7698 Acc: 87.44% | L
R: 0.000125
Epoch [114/200] Train Loss: 0.6204 Acc: 75.85% | Val Loss: 0.7533 Acc: 87.24% | L
R: 0.000122
Epoch [115/200] Train Loss: 0.6382 Acc: 74.21% | Val Loss: 0.7796 Acc: 86.84% | L
R: 0.000120
Epoch [116/200] Train Loss: 0.6006 Acc: 76.91% | Val Loss: 0.7714 Acc: 87.34% | L
R: 0.000118
Epoch [117/200] Train Loss: 0.6379 Acc: 72.30% | Val Loss: 0.7977 Acc: 85.94% | L
R: 0.000115
Epoch [118/200] Train Loss: 0.6079 Acc: 73.43% | Val Loss: 0.7685 Acc: 87.14% | L
R: 0.000113
Epoch [119/200] Train Loss: 0.5869 Acc: 75.14% | Val Loss: 0.7714 Acc: 87.74% | L
R: 0.000111
Early stopping triggered after 119 epochs
Loading best model (Val Acc: 87.94%)
```

```
C:\Users\Park\AppData\Local\Temp\ipykernel_58780\2218515974.py:224: FutureWarnin
g: You are using `torch.load` with `weights_only=False` (the current default valu
e), which uses the default pickle module implicitly. It is possible to construct
malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more
details). In a future release, the default value for `weights_only` will be flipp
ed to `True`. This limits the functions that could be executed during unpickling.
Arbitrary objects will no longer be allowed to be loaded via this mode unless the
y are explicitly allowlisted by the user via `torch.serialization.add_safe_global
s`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
  checkpoint = torch.load(os.path.join(save_path, 'Vit_best_model.pth'))
```

Out[16]: <All keys matched successfully>

Test Phase

In [9]:

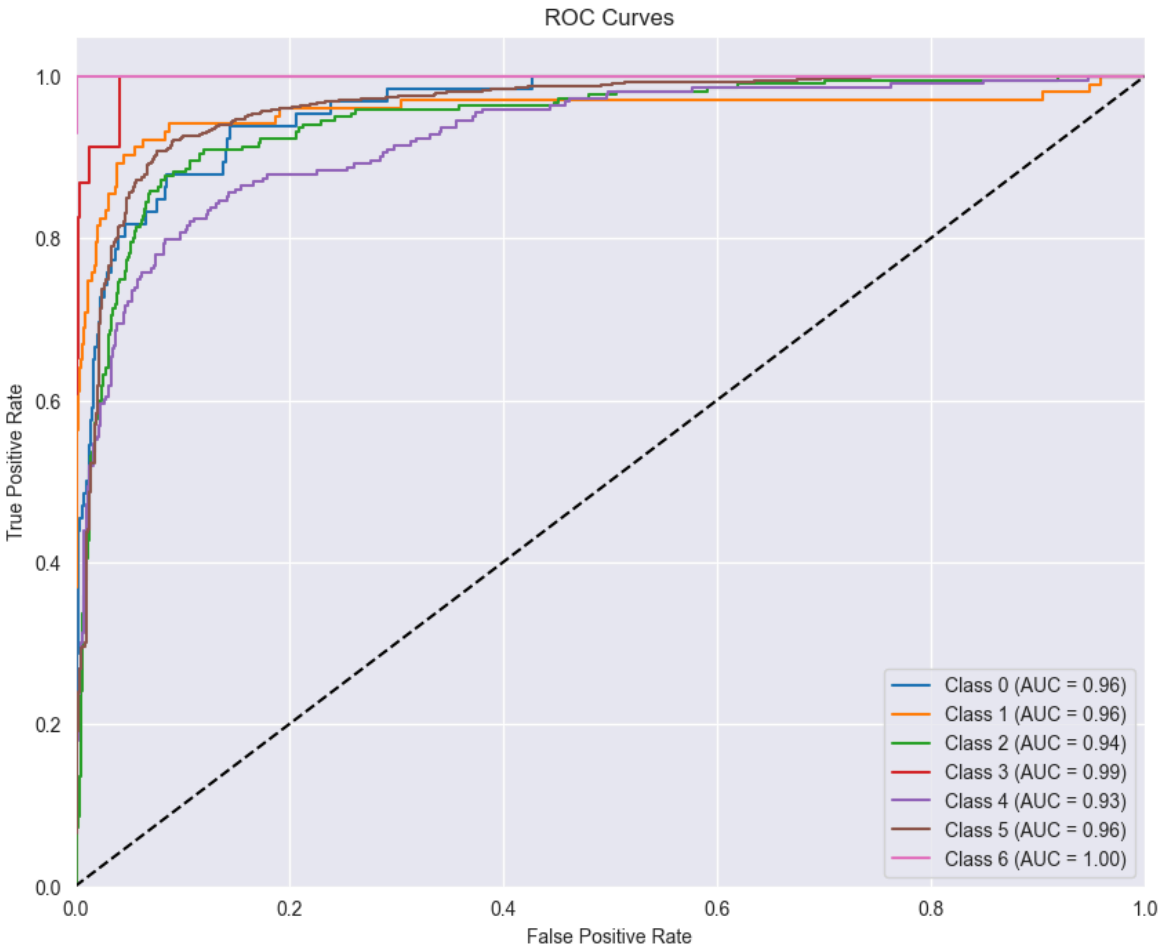
```python
# Load the best model
save_path = 'Skin Disease Transformer'
checkpoint = torch.load(os.path.join(save_path, 'Vit_best_model.pth'))
model.load_state_dict(checkpoint['model_state_dict'])

# Test set evaluation
model.eval()
y_true = []
y_score = []
attention_maps = []
original_images = []

with torch.no_grad():
    for inputs, targets in test_loader:
        inputs, targets = inputs.cuda(), targets.cuda()

        # Get model outputs
        outputs = model(inputs)
        outputs = outputs.softmax(dim=-1)

        # Store original images and results
        original_images.extend(inputs.cpu().numpy())
        y_true.extend(targets.cpu().numpy())
        y_score.extend(outputs.cpu().numpy())

y_true = np.array(y_true)
```

```
y_score = np.array(y_score)
y_pred = np.argmax(y_score, axis=1)

# 1. Calculate performance metrics
accuracy = accuracy_score(y_true, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(y_true, y_pred, avera

print(f'Accuracy: {100*accuracy:.3f}%')
print(f'Precision: {100*precision:.3f}%')
print(f'Recall: {100*recall:.3f}%')
print(f'F1-score: {100*f1:.3f}%')

# 2. Plot ROC curves
plt.figure(figsize=(10, 8))
y_true_bin = label_binarize(y_true, classes=range(n_classes))
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
    plt.plot(fpr[i], tpr[i], label=f'Class {i} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend(loc="lower right")
plt.show()

# 3. Plot confusion matrix
plt.figure(figsize=(10, 8))
cm = confusion_matrix(y_true, y_pred)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100
sns.heatmap(cm_normalized, annot=True, fmt='.1f', cmap='Blues')
plt.xlabel('Predicted labels')
plt.ylabel('True labels')
plt.title('Normalized Confusion Matrix (%)')
plt.show()

# 4. Visualize original images and attention maps
def get_attention_map(model, input_tensor):
    # Function to get attention maps
    model.eval()
    attention_maps = []

    def hook_fn(module, input, output):
        attention_maps.append(output.detach())

    for name, module in model.named_modules():
        if 'attn' in name and hasattr(module, 'qkv'):
            handle = module.qkv.register_forward_hook(hook_fn)

    with torch.no_grad():
        _ = model(input_tensor)

    handle.remove()
```

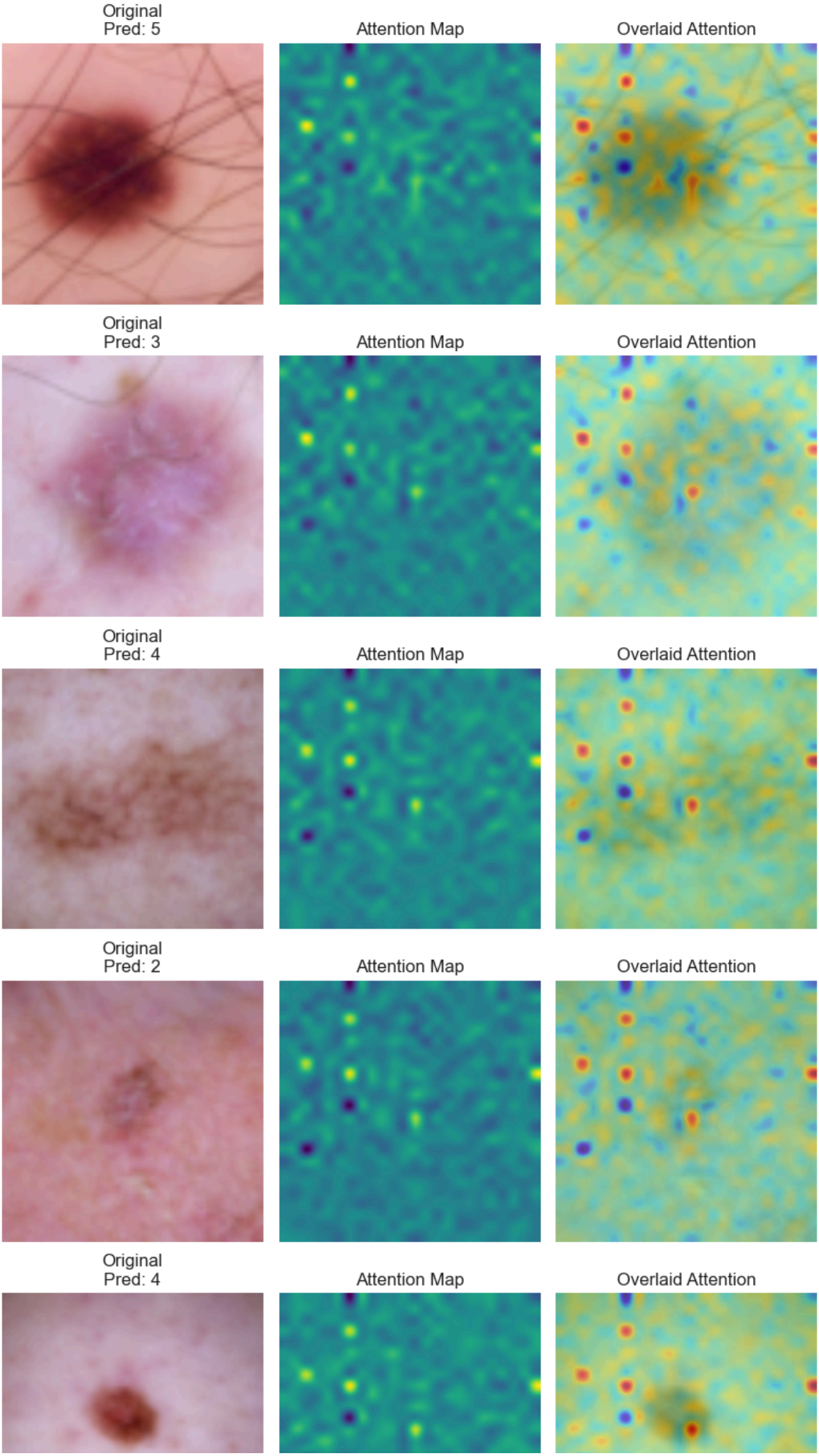
```
    if len(attention_maps) == 0:
        raise ValueError("No attention maps were captured")

    attention = attention_maps[-1]

    try:
        n = int(np.sqrt(attention.shape[-1]))
        attention = attention.mean(dim=1)
        attention = attention.reshape(-1, n, n)
        attention = torch.nn.functional.interpolate(
            attention.unsqueeze(1),
            size=(224, 224),
            mode='bicubic',
            align_corners=False
        ).squeeze()
        return attention.cpu().numpy()
    except Exception as e:
        return None

def visualize_attention(model, original_images, predictions, num_samples=5):
    # Create a large figure with 5 rows and 3 columns
    plt.figure(figsize=(8, 15))

    for idx in range(min(num_samples, len(original_images))):
        # Process original image
        orig_img = original_images[idx].transpose(1, 2, 0)
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        orig_img = std * orig_img + mean
        orig_img = np.clip(orig_img, 0, 1)

        # Get attention map
        input_tensor = torch.FloatTensor(original_images[idx:idx+1]).cuda()
        attention_map = get_attention_map(model, input_tensor)

        if attention_map is not None:
            # Original image
            plt.subplot(5, 3, idx*3 + 1)
            plt.imshow(orig_img)
            plt.title(f'Original\nPred: {predictions[idx]}')
            plt.axis('off')

            # Attention map
            plt.subplot(5, 3, idx*3 + 2)
            plt.imshow(attention_map, cmap='viridis')
            plt.title('Attention Map')
            plt.axis('off')

            # Overlaid attention map
            plt.subplot(5, 3, idx*3 + 3)
            plt.imshow(orig_img)
            plt.imshow(attention_map, cmap='jet', alpha=0.5)
            plt.title('Overlaid Attention')
            plt.axis('off')

    plt.tight_layout()
    plt.show()

# Usage
```

```
# Usage
visualize_attention(model, original_images, y_pred)
```

```
C:\Users\Park\AppData\Local\Temp\ipykernel_75228\3466997327.py:15: FutureWarning:
You are using `torch.load` with `weights_only=False` (the current default value),
which uses the default pickle module implicitly. It is possible to construct mali
cious pickle data which will execute arbitrary code during unpickling (See http
s://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more de
tails). In a future release, the default value for `weights_only` will be flipped
to `True`. This limits the functions that could be executed during unpickling. Ar
bitrary objects will no longer be allowed to be loaded via this mode unless they
are explicitly allowlisted by the user via `torch.serialization.add_safe_globals
`. We recommend you start setting `weights_only=True` for any use case where you
don't have full control of the loaded file. Please open an issue on GitHub for an
y issues related to this experimental feature.
  checkpoint = torch.load(os.path.join(save_path, 'Vit_best_model.pth'))
Accuracy: 85.786%
Precision: 85.660%
Recall: 85.786%
F1-score: 85.641%
```



```
C:\Users\Park\AppData\Local\Temp\ipykernel_75228\3466997327.py:132: UserWarning:
Creating a tensor from a list of numpy.ndarrays is extremely slow. Please conside
r converting the list to a single numpy.ndarray with numpy.array() before convert
ing to a tensor. (Triggered internally at C:\actions-runner\_work\pytorch\pytorch
\builder\windows\pytorch\torch\csrc\utils\tensor_new.cpp:281.)
  input_tensor = torch.FloatTensor(original_images[idx:idx+1]).cuda()
```


Original
Pred: 5
Attention Map
Overlaid Attention
Original
Pred: 3
Attention Map
Overlaid Attention
Original
Pred: 4
Attention Map
Overlaid Attention
Original
Pred: 2
Attention Map
Overlaid Attention
Original
Pred: 4
Attention Map
Overlaid Attention